Índice

1 - Introducimos el código javascript en el documento HTML....1
2 - El lenguaje javascript....2
3 - Donde insertamos el código javascript....3
4 - Palabras reservadas....4
5 - Tipos de datos....4
5 - Variables....6
6 - Operadores....6
7 - Sentencias condicionales y bucles....9

1 - Introducimos el código javascript en el documento HTML

```
<!DOCTYPE html>
<html>
<head>
      <meta http-equiv= "content-type" content="text/html; charset=utf-8">
      <title>Hola Mundo</title>
</head>
<body>
      <script>
            alert( 'Hola mundo en JavaScript');
      </script>
</body>
</html>

<!DOCTYPE html>
<html>
<head>
      <meta http-equiv="content-type" content="text/html; charset=utf-8">
      <title>Hola Mundo</title>
      <script type="text/javascript" src="codigo.js">
      </script>
</head>
<body>
      Hola, esta es mi página
</body>
</html>
```

codigo.js tendría, por ejemplo :

```
alert( 'Hola mundo en JavaScript');
```

Otra forma :

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type"
      content="text/html;charset=iso-8859-1" />
<title>Ejemplo 1</title>
      <script type="text/javascript">
            alert("Prueba de JavaScript en el head ááíóú ñÑ");
      </script>
</head>
<body>
<h1>Ejemplo 1: código embebido</h1>
<script type="text/javascript">
      alert("Prueba de JavaScript en el body ááíóú ñÑ");
</script>
</body>
</html>
```

El código javascript puede estar insertado en atributos de etiquetas HTML.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type"
content="text/html;charset=iso-8859-1" />
<title>Ejemplo 1</title>
</head>
<body>
<h1>Ejemplo 2: código en elementos HTML</h1>
<p> Esto es un párrafo normal </>
<p onclick="alert('Insertando javascript si haces click aquick')">
Cuidado si haces click aqui !!!
</p>

<script type="text/javascript">
      alert("Prueba de JavaScript en el body ááíóú ñÑ");
</script>
</body>
</html>
```

EJERCICIOS :

1) Busca en internet la diferencia entre colocar el código javascript en el head o en el body.
2) Escribe un archivo HTML que muestre un alert en javascript con tu nombre y apellidos. Prueba a hacerlo en el head, body e insertando con src el código. Inserta la instrucción document.write("un texto cualquiera"); antes y después de cada alert.
3) Muestra un mensaje que diga : no hagas click aquí !! utilizando la etiqueta h1 o h2. Al hacer click ahí deberá salir un alert que diga : No haces ni caso

2 - El lenguaje javascript

Es case-sensitive, o sea, distingue entre mayúsculas y minúsculas. No es lo mismo :

```
alert("hola"); // Correcto
```
que
```
Alert("hola"); // Incorrecto
```

Pregunta : ¿ Es lo mismo **alert("hola");** que **alert("Hola");** ?

Comentarios en el código : // para una línea y /* .... hasta .... */ para un bloque de líneas

```
<script type="text/javascript">
      // Este modo permite comentar una sola línea
      /* Este modo permite realizar
comentarios de
varias líneas */
</script>
```

Además, existen estándares como **JSDoc**, que permiten generar documentación automáticamente a partir de comentarios estructurados.

```
/**
 * Calcula el área de un círculo.
 * @param {number} radio - Radio del círculo.
 * @returns {number} Área calculada.
 */
function areaCirculo(radio) {
  return Math.PI * radio * radio;
}
```

JavaScript ignora los espacios, las tabulaciones y los saltos de línea presentes entre los símbolos del código. Es una buena práctica identar y separar el código para mejorar legibilidad y presentación del código.

Punto y coma. Se debe poner punto y coma (;) al final de cada instrucción. Se puede omitir si se escribe una instrucción por línea, sin embargo este método no es recomendable puesto que se pueden producir errores. Funciona :

```
var jose = 4   // no recomendable
var luis = 5   // no recomendable
var i  =  4 ; var j = 5 ;
```

en instrucciones como if (que veremos) y loops el funcionamiento es por instrucción y no por línea con lo que :

```
// Código correcto
if (a == 4)
{
      instrucciones ;
}
```

```
y no
// Código incorrecto. El if ya lleva las llaves que delimitan el alcance.
if (a == 4) ;
{
      instrucciones ;
}
```

3 - Donde insertamos el código javascript

Puede ir en el <HEAD> o en el <BODY>. Aunque es un tema de discusión, se está utilizando comunmente su inclusión en el <BODY>. Se inserta con :

```
<script type="text/javascript">

      // Código javascript
</script>
```

Si la página se declara como XHTML la cosa cambia un poco :

```
<script type="text/javascript">

//<![CDATA[

      // Código javascript

//]]>

</script>
```

Si el código va en un archivo separado con extensión js :

```
<script type="text/javascript" src="myscript.js"></script>
```

Ejercicio : Escribe dos ventajas de hacer el código javascript en un archivo aparte.

Ejercicio : Busca lo que hace **defer** en html !!

4 - Palabras reservadas

Javascript tiene una serie de palabras reservadas que no se pueden utilizar para definir variables, funciones, etc. Estas palabras son :

```
break, case, catch, continue, debugger, default, delete, do, else,
finally, for, function, if, in, instanceof, new, return, switch,
this, throw, try, typeof, var, void, while, with
```

5 - Tipos de datos

Tipos de datos primitivos .Números, cadenas de texto y valores booleanos

Números : Todos se representan a través del formato punto flotante de 64 bits. Es el formato llamado double en Java o C++.
Se pueden utilizar tambien valores hexadecimales escribiendo 0x y luego el número hexadecimal.

Cadenas de texto : Con este tipo de datos (string) es posible representar secuencias de letras, dígitos, signos de puntuación o cualquier otro carácter de Unicode. La cadena de caracteres va entre comillas simples o dobles (" o '). Las cadenas de texto se pueden concatenar (juntar) con el signo +. Ejemplo :

var cadena = “Jose”;
cadena = cadena + “ Luis ”; // cadena contiene ahora “Jose Luis ”
cadena = cadena + 1234 // cadena contiene ahora “Jose Luis 1234”

Secuencias de escape. Se utilizan para representar ciertos signos :

Secuencia de escape	**Resultado**
\'	Comilla simple
\"	Comilla doble
\\	Símbolo \
\n	Nueva línea
\t	Tabulador horizontal
\v	Tabulador vertical
\f	Salto de página
\r	Retorno de carro
\b	Retroceso

Valores booleanos : Almacena valores lógicos, true o false, verdadero o falso.

Ejercicio : Realiza un alert que muestre el siguiente mensaje (observa que son 3 líneas) :

I'm guay
You're “guay” and \
He's guay

5 - Variables

Zonas de memoria de un ordenador que se identifican con un nombre y en las cuales se almacenan datos.
Declaración :

```
var variable1 ;
var variable2;
var variable3, variable4;
```

Inicialización :

```
var variable1 = 30 ; // Por asignación directa
var variable2 = variable1 * 20 ; // Asignación indirecta a través de un cálculo
var variable3 = prompt("Introduce un valor”);  // Solicitando valor a usuarios
```

IMPORTANTE : Se pueden definir variables sin la palabra clave **var**. En ese caso la variable se crea como global, con lo que si la variable ya existe se sustituirá su valor por el de la nueva variable. O sea, que es muy recomendable utilizar la palabra clave **var** para declarar variables. El tema del ámbito de variables lo veremos mas adelante. Puedes adelantarte y ver un ejemplo en :

http://librosweb.es/javascript/capitulo_4/ambito_de_las_variables.html

EJERCICIO :
Crea un script que muestre con un alert o varios tu nombre, apellidos y edad utilizando variables en vez de constantes.

Javascript permite definir dos veces la misma variable, lo que no es muy correcto :
var edad = 10
…..
var edad = 20

Let :

```
let edad = 10;
```

```
if( true ) {
      let edad = 30;
      console.log(edad) ;   ==> Imprime 30
}
console.log(edad) ;    ===> Ahora imprime 10;

let edad = 10;
if( true ) {
      let edad2 = 30;
      console.log(edad2) ;   ==> Imprime 30
}
console.log(edad) ;    ===> Imprime 10;
console.log(edad2) ;   ===> Undefined
```

`const`: declara variables cuyo valor no puede ser reasignado. Es útil para constantes o referencias que no deben cambiar.

El ámbito de las variables define dónde son accesibles. En JavaScript distinguimos:

Global: accesible desde cualquier parte del código.
De función: accesible solo dentro de la función en la que se declara.
De bloque: accesible solo dentro de un bloque delimitado por {} (if, for, etc.).

Cuando declaras una variable con const, lo que no puedes hacer es reasignar la referencia de la variable.
Pero sí puedes modificar el contenido interno del objeto o array al que apunta.

const numeros = [1, 2, 3];

// ✅ Se puede modificar el contenido
numeros.push(4); // [1, 2, 3, 4]
numeros[0] = 100; // [100, 2, 3, 4]
numeros.pop(); // [100, 2, 3]

// ❌ No se puede reasignar la referencia
numeros = [10, 20, 30]; // ERROR: Assignment to constant variable

👉 Ejemplo con un objeto:

const persona = { nombre: "Ana", edad: 25 };

// ✅ Se puede cambiar una propiedad
persona.edad = 26;

// ✅ Se puede añadir una propiedad nueva
persona.profesion = "Ingeniera";

// ❌ No se puede reasignar el objeto entero
persona = { nombre: "Luis" }; // ERROR

6 - Operadores

- Operadores aritméticos.
- Operadores lógicos.
- Operadores de asignación.
- Operadores de comparación.
- Operadores condicionales.

Operadores aritméticos. Pemiten realizara cálculos entre variables numéricas.

+	Suma entre dos operandos
-	Resta entre dos operandos
*	Multiplicación
/	División
%	Divide dos operandos y devuelve el resto
++	Incrementar un valor
--	Decrementar un valor

Ejemplo operador %

```
var anyo = 2014;
var bisiesto = anyo % 4 ;    // Divide 2014 entre 4 y almacena el resto en la
variable bisiesto.
                      // El valor de bisiesto será 0, 1, 2, 3
                  // Si el valor de bisiesto es 0 el 2014 es bisiesto.

var variable1 = 13;
++variable1;   // Variable1 vale ahora 14
variable1++;   // Variable1 vale ahora 15, Parecen iguales, pero no lo son.
Observa el siguiente ejemplo.

var variable1 = 1;
var variable2 = ++variable1 ; // Primero incrementa variable1 y luego la asigna
a variable2
                        // Al final variable1 y variable2 contienen 2
```

sin embargo

```
var variable1 = 1;
var variable2 = variable1++; // Primero asigna la variable1 a la variable2 y
luego la incrementa.
                        // El valor de variable2 es 1 y la da variable1 es 2.
```

EJERCICIO :
Define variables numéricas y realiza operaciones aritméticas entre ellas. Muestra los resultados con alert.

Operadores de asignación. Además del = podemos utilizar otros como :

+=	Suma y asigna
-=	Resta y asigna
*=	Multiplica y asigna
/*	Divide y asigna
%=	Módulo y asigna

Ejemplo :

```
var ingresos = 1000 ;
// Los ingresos aumentan el doble
ingresos *= 2 ;   //   Es lo  mismo que poner     ingresos = ingresos * 2 ;
```

Operadores de comparación. Permiten comparar todo tipo de variables con el fin de verificar sus valores. El resultado de la comparación devolverá un valor booleano (true o false)

<	Si el operando de la izquierda es menor que el de la derecha devuelve true
<=	Si es menor o igual
= =	Si es igual
>	Si es mayor
>=	Si es mayor o igual
!=	Si los dos operandos no son iguales devuelve true.
= = =	Si los dos operandos son iguales y del mismo tipo de datos
!= =	Si operandos diferentes y/o de tipo diferentes devuelve true.

Ejemplos :

```
var a = 1 ;
var b = 5;
var c = a < b ; // c contendrá true
var d = a > b ; // d contendrá false
```

Operadores lógicos. Combina expresiones lógicas con el fin de evaluar si el resultado de una combinación es verdadero o falso. Se suelen utilizar para tomar decisiones.

&&	Y	Verdadero si todos los valores son verdaderos (true)
||	O	Verdadero si al menos uno de los valores es verdadero
!	No	Invierte el valor booleano del operando

Ejemplo :

```
var a = 1 ;
var b = 2;
var c = 3 ;
var d = 4;
var e = a < b && c < 4 ; // e contendrá true
var f = a > b || d > c ; // f contendrá true
```

EJERCICIO :

Teniendo en cuenta los valores de las variables a, b, c y d anteriores indica si el resultado de las operaciones es true o false :

```
j = a <= b && d = = c ;  // la variable j contendrá ? _________
i = c < d && b > a ;      // la variable i contendrá ? __________
m = a <= b || d = = c ;  // la variable m contendrá ? _________
k = c < d || b > a ;      // la variable k contendrá ? __________
```

Se deben utilizar paréntesis cuando unimos expresiones mas complejas. Ejemplo :

```
n = ( ( a > b ) && ( c > d ) ) || d = = a    // En que orden se ejecutan las
comparaciones ?
```

Operadores condicionales. Podemos indicarle al navegador que ejecute una acción después de evaluar una expresión. Consta de tres partes : la expresión a evaluar, la acción a realizar si es verdadera y la acción a realizar si es falsa. El operador es :

expresion ? AccionVerdadero : AccionFalso ;

Ejemplo :

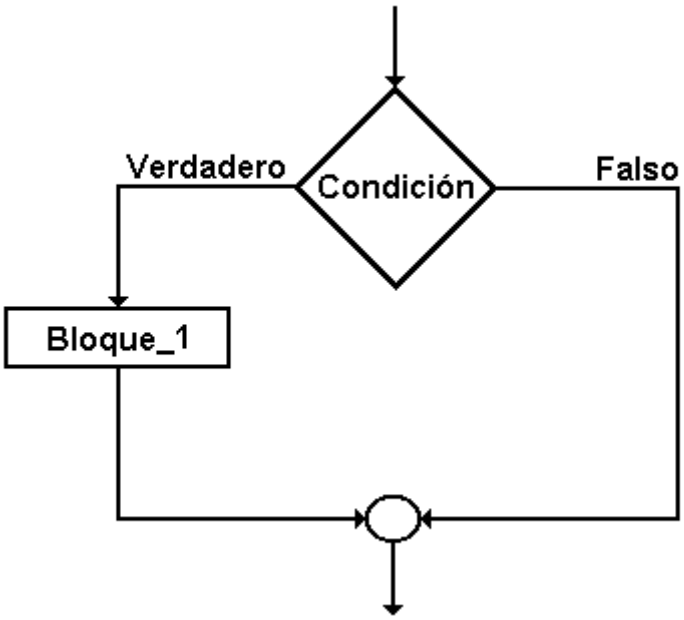
```
<script type="text/javascript">
      var a = prompt("Introduzca un número");
      a <= 5 ? alert("menor que 5") : alert("mayor o igual que 5");
</script>
```

7 - Sentencias condicionales y bucles

if , switch
while, for (bucle)

if ==> Permite tomar decisiones. Formato1 :

```
if(expresion) {
        instruccion1;
        instrucción2;
        …...
}
instruccionA;
instrucciónB;
…..
```



En este formato, si la expresión es verdadera se ejecutan instrucción1, instrucción2 y siguientes, al terminar sigue con instruccionA, instruccionB y siguientes.
Si la expresión anterior es falsa, no se ejecutan las instrucciones entre llaves y pasa a ejecutar instrucciónA, instruccionB y siguientes.

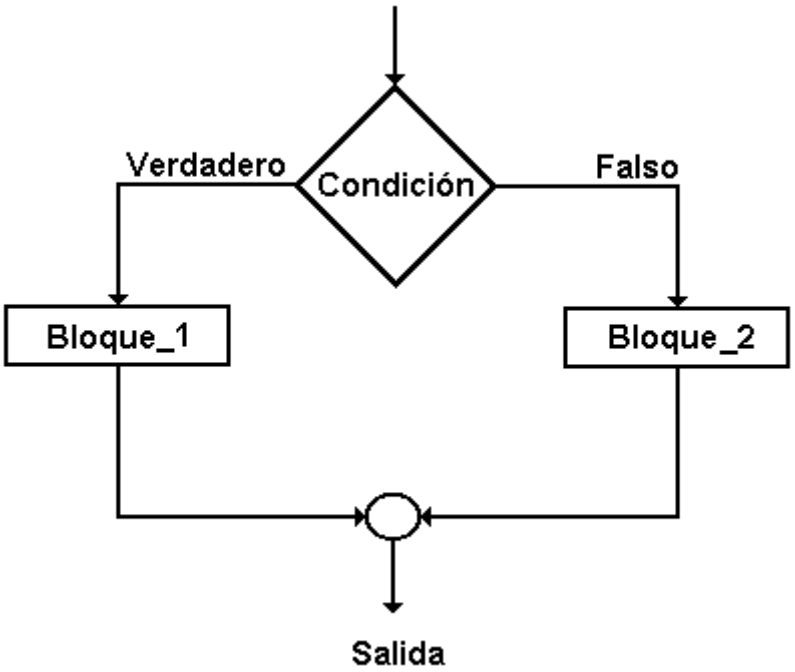
En caso de una instrucción sola podemos omitir las llaves. Ejemplo :

```
if( a < 5) alert("a menor que cinco”);
```

Formato2 :

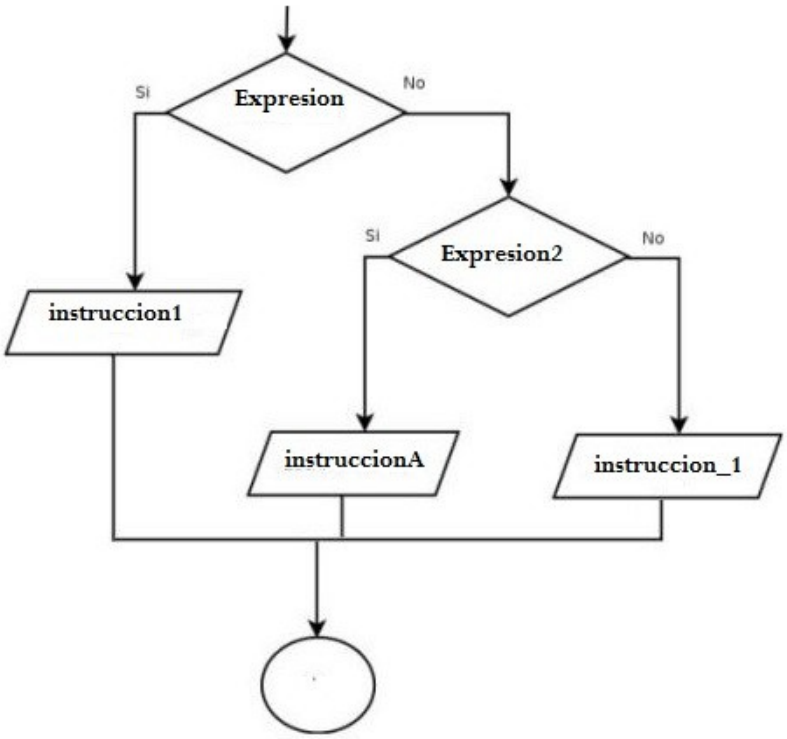
```
if(expresion) {
        instruccion1;
        instruccion2;
} else {
        instruccionA;
        instruccionB;
}
```

En este formato se indica que si la expresión es verdadera se ejecutarán las instrucciones que terminan con un número y en caso contrario (else) se ejecutarán las que terminan el letra.



Formato3 : Se complica un poco el if con la expresion else if, de todas formas lo ponemos aunque se recomienda cuando hay muchos if anidados que se utilice la sentencia switch en aras de una mejor comprensión y seguimiento del código.

```
if(expresion) {
        instrucciones1;
        instrucciones2;
} else if (expresion2) {
        instruccionesA;
        instruccionesB;
        }
        else {
        instrucciones_1;
        instrucciones_2;
        }
```

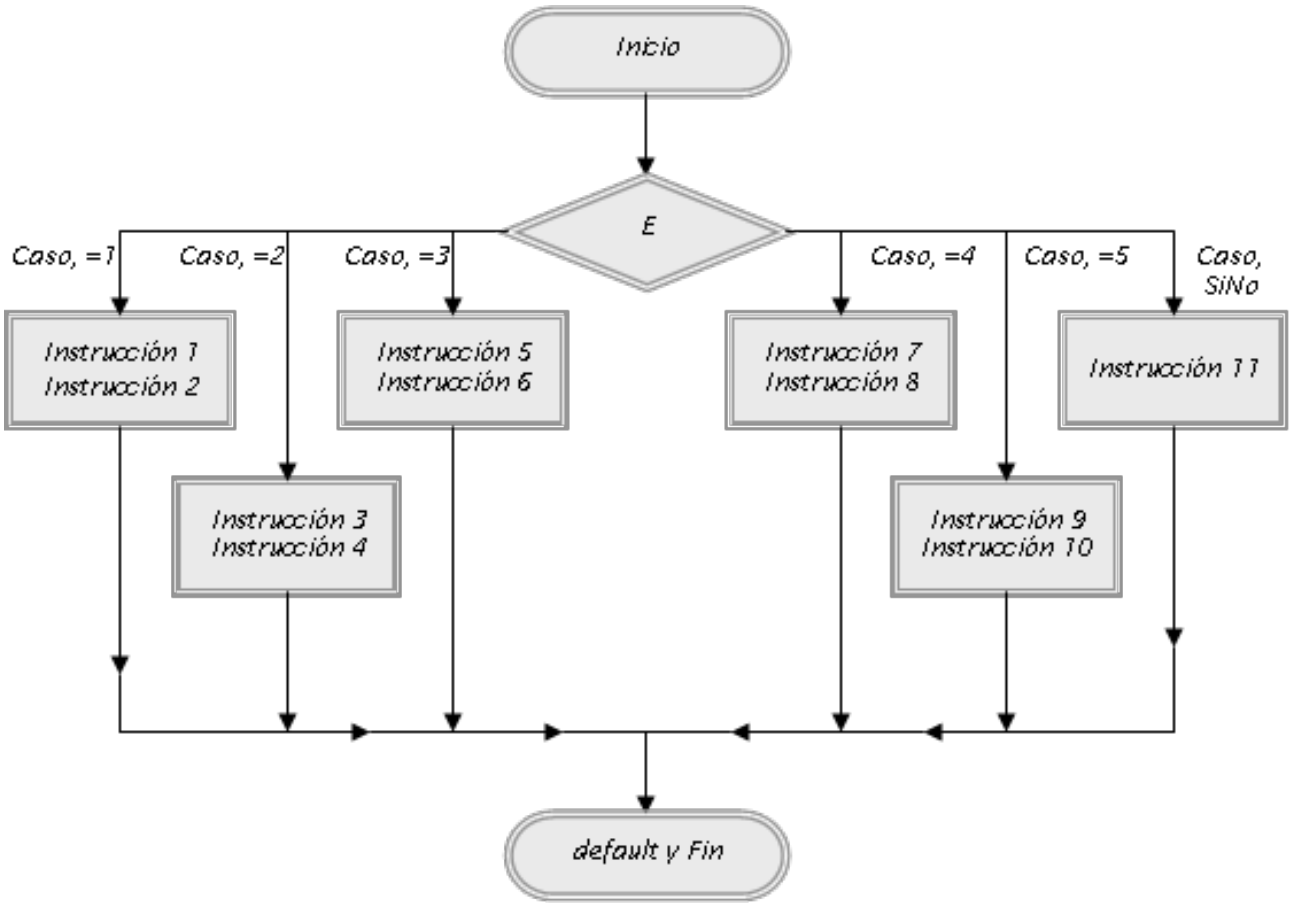


switch ==> Permite comparar el valor de una variable con algunos valores conocidos.

Formato :

```
switch ( expresion) {
        case valor1:
                instrucciones si expresion es igual a valor1 ;
                break;
        case valor2:
                instrucciones si expresion es igual a valor2;
                break;
        default:
                instrucciones si expresion no es ninguno de los anteriores;
}
```

Diagrama de flujo del switch :



while ==> El navegador ejecutará instrucciones hasta que se cumpla una condición

Formato :

```
while (expresion) {
        instruccion1;
        instruccion2;
}
```

Se evalua la expresión. Si es verdadera, se ejecutan las instrucciones. Al terminar se vuelve a evaluar la expresión. De esta forma se ejecutarán las instrucciones mientras la expresión sea verdadera.

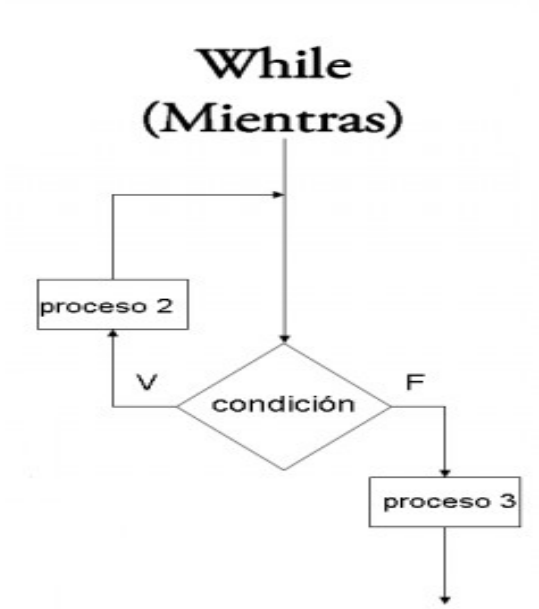
Pregunta : ¿ podría darse el caso de que nunca se ejecuten instruccion1 e instruccion2 en el bucle while ?

Una variación de este bucle es :

```
do  {
        instruccion1;
```

```
        instruccion2;
} while (expresion)
```

Pregunta : ¿ podría darse el caso de que nunca se ejecuten instruccion1 e instruccion2 en el bucle do while ?



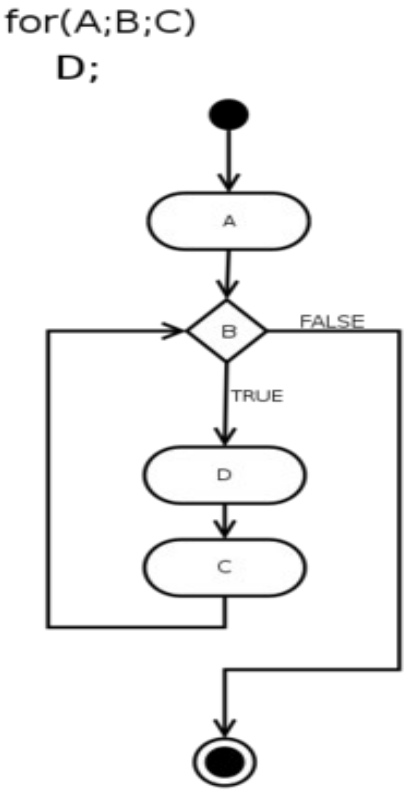
Ejercicio : Haz el diagrama de flujo del bucle do... while

for ==> Bucle que permite que el navegador ejecute las instrucciones que se encuentran dentro hasta que la expresión sea false.

Formato :

```
for ( valor_inicial ; expresion_condicional ; incremento_decremento ) {
        instruccion1;
        instruccion2;
}
```

Diagrama del flujo del bucle for :



Ejemplo : Mostramos los 100 primeros números ¿ o no ?

```
for ( i = 0 ; i < 100 ; i++ ) {

      document.write("Número : " + i );

}
```

EJERCICIOS:

- Pide un número del 1 al 7 (prompt) y muestra el día de la semana siendo 1 lunes, 2 martes.... hazlo dos veces, una con if y otra con switch. Si el número no está entre 1 y 7 muestra un mensaje de error.
- Visualiza los 100 primeros números pares utilizando bucle while.
- Visualiza los 100 primeros números impares utilizando bucle for.

Nota para ejercicios de la unidad :
Si el usuario pulsa cancelar el valor introducido es null

```
      var a = prompt("Introduzca un número");
      if ( a !== null ) {
```

Se puede preguntar con **confirm** si desea continuar o no.